Final Project

OBDI Telematics Interface, Corvette 86

Team 9

Samuel Cumens, Danny El Rayes, Preston McCubbin,

Sach Sunuwar

CSE 525

Abstract

This project is designing a real-time telematics dashboard for a pre-OBDII 1986 Corvette using the GM ALDL (Assembly Line Diagnostic Link) protocol. The system that was created captures raw diagnostic data from the ECU, processes it and displays on a dashboard. Firstly the signal is decoded using an Arduino program interpreting the pulse-width. Then next processes the data into meaningful parameters such as engine speed, coolant temperature, throttle position, and many other sensor readings. Traditional serial communication methods could not be used here due to it being pre-OBDII. Instead, the signal was analyzed at the microsecond level allowing binary data to be reconstructed from pulse durations. The decoded data frames were then transmitted to the rest of the system through serial, where it was parsed and served through a Flask based web interface. This project demonstrates the integration of embedded systems, signal decoding/processing, and web interfacing.

Body

Setup/ Signal gathering:

The first part of this project would require receiving the ALDL (Assembly Line Diagnostic Link) signal from the car, this signal needed to be gathered to decode using a Byte Mask from sources online to convert into readable values. At first it was misunderstood what the signal would look like, It was thought that the signal would come out in frames, in which would go through a simple circuit of resistors and transistors, then to a FT232RL USB-serial adapter which would carry it into the Raspberry Pi. However after deeper research, it was learned that the ALDL signal does not follow a conventional serial communication format, therefore an Arduino was needed to interpret instead. It was thought that the signal consisted of how long the signal stayed at a level, and whether it was High or Low. The ALDL is 160 baud, therefore T=1/160 = 0.00625s = 6250μs, which would be one real bit. Through research online and by observing patterns printing through terminal such as:
H4356 + L1936 ≈ 6250 μs
L1936 + H4324 ≈ 6250 μs

It was determined that each complete bit occupies about 6250 microseconds, corresponding to a data rate of about 160 baud. This helped understanding the timing-based encoding the signal uses. After trying to combine HIGH and LOW durations to determine each bit, an old blog post was found of someone doing a similar project, where it was found that the LOW pulse duration alone is sufficient to determine the bit value. Specifically:

- ~1900 μs , one bit value = 0
- ~5900 μs , the other bit value = 1

This made the decoding significantly simpler and better to grasp, by removing the need to pair signals and calculate full bit periods. Finding this information was extremely helpful, as it also explained how the decoded bits are structured into 9 bits, 1 start bit which is always 0, and 8 data bits after. These units formed complete data frames. This helped a lot to correctly align

and get meaningful bytes from the data stream. Also ours had to be inverted due to our transistor in our circuit. Printing frames out continuously while the car key was On, this was received:
000000000 (00 / 0) 000000000 (00 / 0) 000000000 (00 / 0) 001111111 (7F / 127) 110001000 (88 / 136) 010001001 (89 / 137) 010111001 (B9 / 185) 010100100 (A4 / 164) 000010100 (14 / 20) 010000000 (80 / 128) 000000000 (00 / 0) 000000000 (00 / 0)000000110 (06 / 6) 110100000 (A0 / 160) 000011001 (19 / 25) 100000000 (00 / 0) 000000000 (00 / 0) 000000000 (00 / 0) 000000000 (00 / 0) 000000000 (00 / 0) 0000

After analyzing all the frames, it was noticed that 111111111 only occurred again after 25 Bytes. I knew that the byte length of the frame I should be expecting is 25 from research. 111111111 occurred at the start of each frame, and this could be seen through the whole terminal. So, that means that the last 24 bytes followed 111111111, therefore the code aligned to 111111111 to create a full frame. Knowing this, the code essentially waits for an edge change and then watches for FF or 111111111, in which it locks and records the 24 bytes after. There is also a gap between frames, in which the frame resets and FF is unlocked if it's determined to be in that Quiet Period. When signal goes HIGH, a LOW pulse just ended, LOW pulse length determines the bit value, then we compare the pulse length interval Vs. minReadableOne and minReadableZero.

```
// reset on gap between frames
   if (currentState == LOW && interval > quietTimeBetween)
   {
     lockFF = false;
     bitGroupIndex = 0;
     frameIndex = 0;
     continue;
   }

   if (currentState == HIGH) //when signal goes HIGH a LOW pulse just ended, LOW pulse length determines the bit value
   {
     char bitChar;

     // invert because of the circuit we have
     if (interval >= minReadableOne)
     {
       bitChar = '0';
     } else if (interval >= minReadableZero)
     {
       bitChar = '1';
     } else {
       continue;
     }
```

These hexadecimal frames are sent through serial to the next section.

Frame Conversion:

To handle the data output by the Arduino, a C program was written to process the incoming data. This was accomplished by creating custom structs to hold the data for the individual hex value pairs defined for the Corvette. Structs are either a bitmap of up to 8 bits, a word left as a binary value, or a numerical value stored as either a float or an integer with an applied conversion to extract the proper value. There were 25 total messages sent, each a byte in length or two hex values. These values were treated as strings within the C code due to the serial communication, which was accomplished with termios, a C library that allows for serial communication similar to opening and writing to or reading from a file.

Multiple flags and options must be set in order for the serial communication to work. Initially, the serial port must be opened using open(), along with a flag, O_RDWR, to determine if the serial port is to be read from or written to, similar to the 'r' or 'w' options used with fopen. A termios struct must then be created and associated with the serial port before the individual flags are set. These include enabling the serial port for reading, ensuring it is a local connection, disabling parity, ensuring there is one stop bit, clearing the size mask, and ensuring that the bytes transferred are 8 bits in size. Disabling echoing to the terminal is also required, as well as ensuring that the data being transferred is treated as raw data and not interpreted as a shutdown command such as Ctrl+C. Additionally, the flags ensure that the code waits indefinitely for a signal, as the program may have to wait to receive data due to the car being off. Finally, the most important flag to be set was the ICANON flag, which allowed the program to read the serial line by line. Due to the way the Arduino code sent information through serial communication, this was paramount to ensuring that the program was properly reading the data. Due to their low-level nature, these flags must be set using bitwise AND and OR operations rather than simply assigning a value, with the exception of VMIN and VTIME, which tell the program to wait for at least 1 byte of data before reading and to wait indefinitely for data.

After the appropriate flags have been set, the code creates a buffer to hold the data. The buffer was set to a size of 256, as it is recommended to have at least twice the space needed, rounded up to the nearest power of two. This program requires at least 74 bytes to hold the data: 2 bytes for each hex value, since they are stored as characters, and 24 spaces to separate the hex values. For troubleshooting purposes, the program outputs the contents of the buffer, then creates a buffer to hold the hex values and a buffer to hold the translated binary number, also stored as a string. For each of the 25 values, a respective function is called to translate the hex values into binary, which is then passed to another function that translates the value appropriately.

For the vast majority of values, translation followed one of two conventions. The first applies to values that exist as a bitmap, where each individual bit must be set individually. For example, here is the first value, named one within the code, being translated:

```
struct one *handle_one(char byte[9]){
   struct one *one = malloc(sizeof(struct one));
   if(one==NULL){
      fprintf(stderr,"%s: could not allocate memory for one; %s\n", "one", strerror(errno));
      exit(-1);
   }
   if(byte[0] == '1'){ one->OVERDRIVE = 1; } else { one->OVERDRIVE = 0; }
   if(byte[1] == '1'){ one->MALF = 1; } else { one->MALF = 0; }
   if(byte[2] == '1'){ one->ref_pulse = 1; } else { one->ref_pulse = 0; }
   if(byte[3] == '1'){ one->ALDL_mode = 1; } else { one->ALDL_mode = 0; }
   if(byte[4] == '1'){ one->dia_pos = 1; } else { one->dia_pos = 0; }
   if(byte[5] == '1'){ one->aldl_pos = 1; } else { one->aldl_pos = 0; }
   if(byte[6] == '1'){ one->high_bat_volt = 1; } else { one->high_bat_volt = 0; }
   if(byte[7] == '1'){ one->shigt_light = 1; } else { one->shigt_light = 0; }
   return one;
}
```

Each bit in the value, represented as a char, is checked for the value '1'. If the bit is set, the corresponding field in the struct is set to 1 before the struct is returned. Values one, twelve, thirteen, fourteen, fifteen, sixteen, and eighteen all follow this convention, though not all use all eight bits. The second convention applies to values that represent a numerical quantity, handled as follows:

```
struct four *handle_four(char byte[9]){
   struct four *four = malloc(sizeof(struct four));
   if(four==NULL){
      fprintf(stderr,"%s: could not allocate memory for four; %s\n", "four", strerror(errno));
      exit(-1);
   }
   int step = 0;
   for(int i = 0; i < 8; i++){
      if(byte[i] == '1'){
         step |= (1 << i);
      }
   }
   four->step_count = step;
   return four;
}
```

The exceptions to both conventions are values seventeen, twenty-two, twenty-three, twenty-four, and twenty-five. Value seventeen, which represents the MAT_TEMP values, has a special function. Because these values are directly mapped to a lookup table, a special interpolation function had to be implemented for cases where a raw value did not exactly match

a table entry. This function used a two-dimensional lookup table, and when a value fell between two entries, the following linear interpolation formula was applied:

$y = y_0 + (raw - x_0) \times (y_1 - y_0) / (x_1 - x_0)$

where (x_0, y_0) and (x_1, y_1) are the two nearest surrounding points in the lookup table, and raw is the value decoded from the message. Additionally, values twenty-two, twenty-three, twenty-four, and twenty-five all simply return the bit mask, as these are the lower and upper bytes of AFR and IBPW respectively.

After the values had been translated and handled accordingly, the corresponding values in each struct were written to the output file and the structs freed. The values were written in the form COOLANT_TEMP: 68, where COOLANT_TEMP is the field name held within the struct and 68 is the value assigned by the corresponding handle_ function. This process was repeated for all twenty-five values sent by the ALDL.

Reading Ouput.txt:

The Python code responsible for reading the C program's output processes the file line by line. Since each value is written in a key: value format, the code is able to split each line and store the key and value accordingly. Because the output file stores plain text, values are converted to integers or floats where necessary so they can be used in calculations and displayed correctly. Additionally, binary values such as 0 or 1 are translated into human-readable labels like "on" or "off" rather than being left in their raw form.
The processed data is stored in a dictionary, with field names normalized for consistency — for example, RPM becomes rpm and MPH becomes speed. Flask then uses this processed dictionary to serve a JSON response through a dedicated data route. On the frontend, the dashboard periodically requests this data and updates the display accordingly, allowing the web interface to show real-time sensor values from the vehicle.

Web Interface:

The web interface was built using Flask, HTML, CSS, and JavaScript. Flask serves as the backend server and provides the main routes for the system, which include the login page, dashboard page, trip logs page, trip summary page, and the `/data` route used for live sensor updates. The purpose of the web interface is to take the decoded vehicle data and display it in a way that is easy to read from a phone or laptop.

The decoded ALDL values are passed through the Python web application after being converted into readable sensor values. These include RPM, coolant temperature, throttle position, battery voltage, and vehicle speed. Flask sends this data to the frontend as JSON

through the `/data` route. On the frontend, JavaScript repeatedly requests new data and updates the dashboard automatically without requiring a page refresh, allowing the dashboard to function as a live telemetry display.

The dashboard also calculates and displays derived metrics such as estimated engine load, battery health, and coolant status, which help present the raw sensor data in a more meaningful way. For instance, rather than displaying only the raw battery voltage, the interface labels the battery condition as charging, okay, or low. Similarly, coolant temperature is categorized as warming up, normal, or hot.

A basic login system was implemented to restrict access to the dashboard, requiring users to authenticate before viewing vehicle data. Once logged in, users can access the live dashboard, view recent trip logs, and view trip summaries. The trip logs page displays saved records from the database, while the trip summary page shows calculated statistics such as average RPM, maximum coolant temperature, average battery voltage, maximum speed, and average engine load.

The web application was also designed to be accessible from multiple devices on the same network. By running Flask on `0.0.0.0`, the dashboard can be opened from a phone or laptop by entering the host machine's IP address followed by port 5000. This makes the system more practical, as users do not need to interact directly with the Raspberry Pi to view vehicle data.

Analysis

Since the OBD1 format tends to differ from car to car, finding the specific information for the 1986 Corvette C4 that this project was focused on presented a significant challenge. The project also faced difficulties in how values were transmitted; for example, each value sent to the Arduino was preceded by a 0xFF byte to mark the start of a new message. While there are many open source projects containing similar decoding techniques, most are specific to an individual vehicle and therefore required adaptation to work with the particular car being decoded.

Additionally, because each message was either a bitmap or a numerical value with its own specific format, the convert.c code required individual handling logic for each of the 25 messages. While these could be partially abstracted using a function prototype as described in

the frame conversion section, the overall pipeline runs relatively slowly by nature, as data must be read from the car, transferred to the Arduino, translated, and then fetched by a web application for display. The volume and variety of messages sent by the car made the translation code particularly substantial, requiring individual structs to contain each message's values as well as individual functions to decode them into a human-readable format.
Working with older hardware presented a significant challenge overall, due to the combination of sparse documentation and rigid, inflexible message formats.

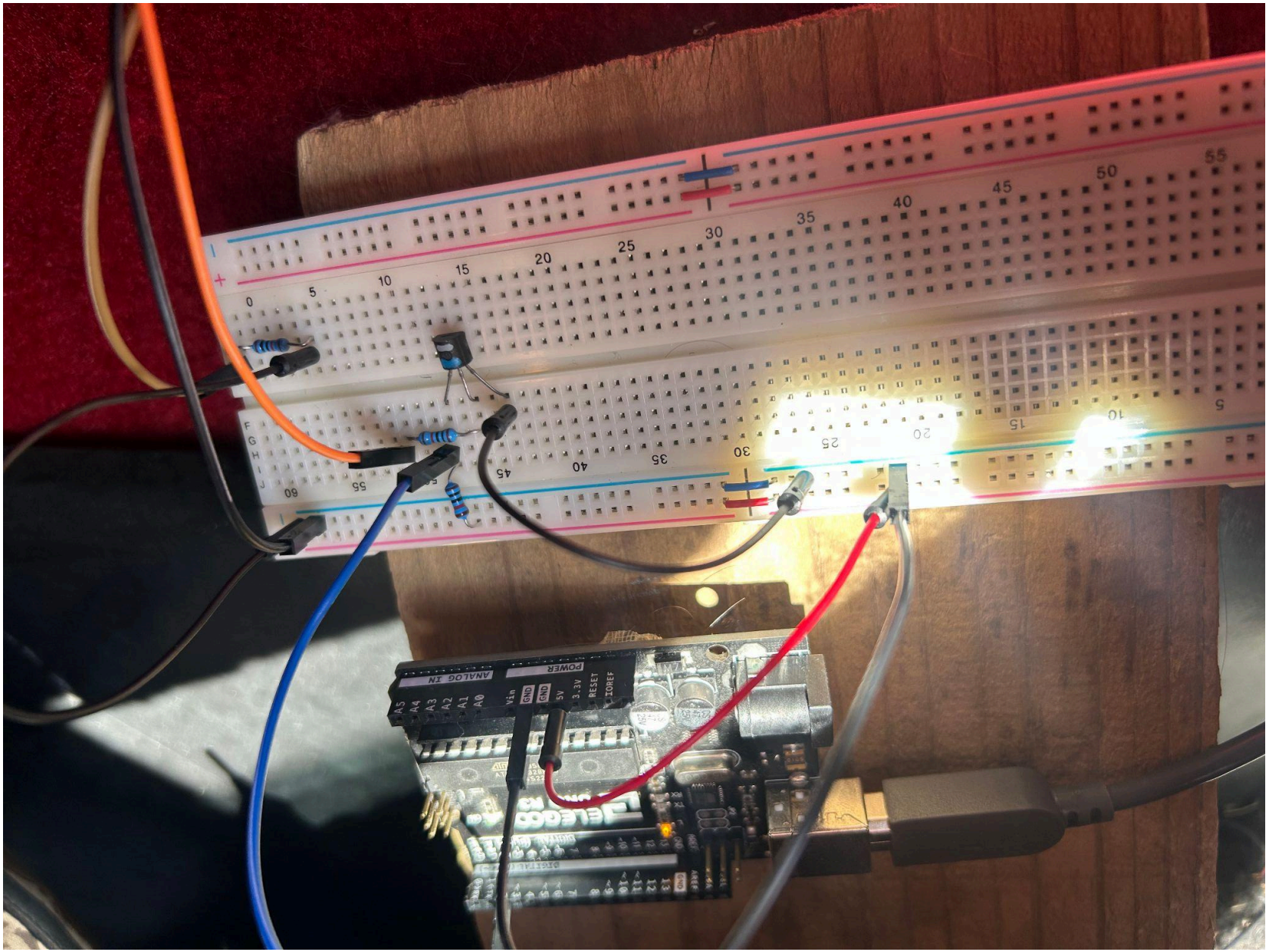
Conclusion

This final project showed how to extract real time data values from an ECU on a pre-OBDII 1986 Corvette using the GM ALDL protocol, and visualize it on a web interface. By decoding pulse-width signals and converting them into meaningful values, the system combines legacy vehicle systems and modern web-based interfaces. This has practical applications in the field particularly it shows how possible similar techniques can be applied to other engineering areas, such as industrial monitoring and IoT systems, where legacy signal interpretation and real-time data visualization are required.

Source Code (Software)

https://github.com/SamuelCooper1105/FP9/tree/main

Schematics (Hardware)


ANALOG IN
POWER
A5
A4
A3
A2
A1
A0
Vin
GND
GND
5V
3.3V
RESET
IOREF

Demo Link

https://youtu.be/M-wEYTzgzmc

Sources:

https://man7.org/linux/man-pages/man3/termios.3.html
https://blog.mbedded.ninja/programming/operating-systems/linux/linux-serial-ports-using-c-cpp/
https://wiki.control.fel.cvut.cz/pos/cv5/doc/serial.html
https://blog.nelhage.com/2009/12/a-brief-introduction-to-termios-termios3-and-stty/